

Plano de Execução — Teste Prático: Portfolio Analytics API (.NET 8)

Documento de apoio: leitura estratégica do teste, decisões técnicas justificadas, algoritmos detalhados e plano de execução em fases.

1. Leitura estratégica do teste

Critério	Peso	Tradução prática
Algoritmos	40%	Fórmulas corretas, unidades corretas , edge cases tratados
Qualidade de código	30%	SOLID pragmático, services pequenos e testáveis
Testes	20%	Os 5 obrigatórios + fronteiras dos edge cases
Documentação	10%	README com premissas explícitas — nota "barata" de ganhar

Sinais escondidos no enunciado (leia como requisitos):

1. **"O seed pode ter inconsistências propositais"** → tratamento defensivo (símbolo sem asset, preço zero, target que não soma 100%) é parte da prova. Cada guarda dessas deve ter um teste e uma linha no README.
2. **O FAQ pede "documente sua decisão" três vezes** → o avaliador quer ver que você sabe que fez uma escolha (normalizar targets, volatilidade null, base de anualização), não que acertou "a" resposta.
3. **"Dê preferência a implementar os algoritmos"** → não usar MathNet/lib de estatística para desvio-padrão. Implemente em uma classe `FinancialMath` própria, com comentários.
4. **3-4 horas** → o escopo é deliberadamente enxuto. Multi-projeto, CQRS, MediatR, AutoMapper = tempo perdido e sinal de má calibração. Cite no README que foi decisão consciente manter simples.
5. **"Não modificar SeedData.json"** → todas as correções de inconsistência acontecem em código/memória, nunca no arquivo.

2. Decisões técnicas (com justificativa)

2.1 Tipos numéricos: `decimal` para dinheiro, `double` para estatística

- Valores monetários, quantidades × preços, pesos e custos → `decimal` (precisão exata em base 10, padrão para finanças em .NET).

- Volatilidade, Sharpe, anualização → `double`, porque `Math.Pow` e `Math.Sqrt` não existem para `decimal`.
- Converter na fronteira (`double`) ao entrar na estatística, (`decimal`) ao voltar) e **arredondar apenas no DTO** (2 casas, `MidpointRounding.AwayFromZero`), nunca em cálculos intermediários.

2.2 Arquitetura: pragmática — 1 projeto API + 1 projeto de testes

- **Controllers finos:** buscam o portfólio, retornam 404 se não existir, delegam ao service, mapeiam DTO. Zero lógica.
- **Services puros:** `PerformanceCalculator`, `RiskAnalyzer`, `RebalancingOptimizer` recebem `Portfolio` + `IReadOnlyList<Asset>` + `MarketData` como parâmetros e **não conhecem EF/DbContext**. É a decisão que mais paga: os testes unitários viram construção de objetos em memória, sem mock de banco.
- **Interfaces** (`IPerformanceCalculator` etc.) registradas no DI → DIP demonstrado sem cerimônia.
- **Limiar de negócio centralizado:** classe `AnalyticsRules` (ou `IOptions`) com `RebalanceThreshold = 2m`, `MinTradeValue = 100m`, `TransactionCostRate = 0.003m`, `HighPositionPct = 25m`, `MediumPositionPct = 15m`, `HighSectorPct = 40m`, `MediumSectorPct = 25m`, `TradingDaysPerYear = 252`. Facilita testes de fronteira e mostra OCP.
- **Reuso:** a volatilidade é usada em Performance e em Risk (Sharpe). Extraia para `FinancialMath` estático (retornos diários, desvio-padrão amostral, anualização) — DRY visível.
- **Rejeitado conscientemente** (citar no README): Clean Architecture multi-projeto, MediatR, AutoMapper, repositório genérico. Custo sem retorno neste escopo.

2.3 Dados: EF Core InMemory + seed no startup

- `DataContext : DbContext` com `DbSet<Portfolio>` e `DbSet<Asset>` (a espec pede exatamente isso).
- Seed no `Program.cs`: ler `SeedData.json` com `System.Text.Json` e `PropertyNameCaseInsensitive = true` (evita o bug silencioso de tudo vir zero/null por casing).
- `MarketData` (Selic, Ibovespa) é um snapshot único: exponha via `IMarketDataProvider` singleton em vez de tabela de 1 linha — mais limpo e mostra DIP de novo.
- `SeedData.json` com `CopyToOutputDirectory=PreserveNewest` e leitura via `AppContext.BaseDirectory`.

2.4 Contratos: DTOs como `record`, separados das entidades

Entidades modelam o seed; `record`s de resposta modelam o JSON da espec (nomes idênticos aos exemplos, camelCase é automático no ASP.NET). Nunca serialize a entidade.

2.5 Erros e nulls

- Portfólio inexistente → `404` com `ProblemDetails` (padrão do ASP.NET, sem esforço extra).
- `volatility`, `annualizedReturn` e `sharpeRatio` são anuláveis por contrato — o próprio FAQ manda retornar `null` quando não há como calcular.

2.6 Convenções financeiras — as premissas que vão para o README

Item	Convenção adotada	Justificativa
Retorno diário	Aritmético: $\frac{Pt}{Pt-1} - 1$	Padrão para vol de curto prazo; documentar (alternativa log-return)
Desvio-padrão	Amostral (divisor $n-1$)	Amostra de 30 dias, não população; documentar
Anualização da vol	$\sigma_{\text{diária}} \times \sqrt{252}$ (dias úteis)	O exemplo do enunciado (15,67) só bate com vol anualizada ; 252 é o padrão de mercado
<code>TotalReturn</code> na fórmula de anualização	Em fração (0,085), não em %	Armadilha nº 1 do teste — $(1 + 8,5)^{\dots}$ explode
"dias" no annualized return	Janela do histórico (30), salvo data explícita no seed	O seed não traz data de compra; premissa documentada
Sharpe	$\frac{(\text{retorno anualizado} - \text{Selic anual})}{\text{vol anualizada}}$	Unidades consistentes dos dois lados; vol 0 ou null → Sharpe <code>null</code>
Vol do portfólio	Série diária do valor agregado ($\sum qty \times \text{preço}_t$) usando só ativos com histórico	Captura correlação implícita; logar/informar cobertura parcial (só 5 de 15 ativos têm histórico)
TargetAllocation ≠ 100%	Normalizar proporcionalmente + <code>LogWarning</code>	Rebalanceamento contra alvo que não soma 100 gera trades sem sentido
Quantidades de ações	Inteiras (<code>floor</code>), sem fracionário	B3 não tem fração no lote padrão; desvio residual aceito e documentado
Preço atual inválido (0/ausente)	Fallback: último preço do histórico → senão <code>AveragePrice</code> , com warning	Melhor que zerar a posição e distorcer o retorno

3. Algoritmos por endpoint

3.1 GET /api/portfolios/{id}/performance

Pipeline:

1. $\text{investedAmount}_i = \text{Quantity} \times \text{AveragePrice}$; $\text{totalInvestment} = \sum$ (ou o campo do seed — se divergirem, use o campo e logue a diferença: possível inconsistência proposital).
2. $\text{currentValue}_i = \text{Quantity} \times \text{CurrentPrice}(\text{resolvido com fallback})$; $\text{currentValue} = \sum$.
3. $\text{totalReturn} = (\text{currentValue} - \text{totalInvestment}) / \text{totalInvestment} \times 100$ — guard: $\text{totalInvestment} \leq 0 \rightarrow \text{null}$.
4. $\text{annualizedReturn} = ((1 + \text{totalReturn}/100)^{(365/\text{dias})} - 1) \times 100$ — guard: $1 + r \leq 0 \rightarrow \text{null}$ (raiz de número negativo).
5. Volatilidade: montar série $(v_t = \sum \text{qty}_i \times \text{preço}_i, t)$ (só posições com histórico completo) → retornos diários → desvio-padrão amostral → $\times \sqrt{252} \times 100$. Menos de 2 pontos → null .
6. Por posição: $\text{return}_i = (\text{CurrentPrice} - \text{AveragePrice}) / \text{AveragePrice} \times 100$ (guard $\text{AveragePrice} = 0$); $\text{weight}_i = \text{currentValue}_i / \text{currentValue} \times 100$.

Edge cases desse endpoint:

Caso	Tratamento
$\text{totalInvestment} = 0$	$\text{totalReturn} / \text{annualizedReturn} = \text{null}$
$\text{AveragePrice} = 0$ em posição	return da posição = null (ou 0 documentado)
Sem histórico ou 1 ponto só	$\text{volatility} = \text{null}$
Preço 0 dentro do histórico	Pular o par de retorno (divisão por zero) e logar
Symbol sem Asset correspondente	Excluir do cálculo + warning; reportar no log
$\text{CurrentPrice} = 0$	Fallback (histórico → AveragePrice) + warning

3.2 GET /api/portfolios/{id}/risk-analysis

1. **Sharpe:** reusa retorno anualizado e vol anualizada do calculator (injeite `IPerformanceCalculator` no `RiskAnalyzer` ou compartilhe `FinancialMath`). $\text{vol null}/0 \rightarrow \text{sharpeRatio} = \text{null}$.
2. **Concentração:** pesos por valor atual → maior posição; top-3 = soma dos 3 maiores pesos.

3. **Setores:** join `(Position.Symbol → Asset.Sector)`, agrupar, somar pesos; setor de asset desconhecido → `"Unknown"`.
4. **Classificação** (regras do enunciado): Alto se posição > 25% OU setor > 40%; Médio se posição em 15-25% OU setor em 25-40%; Baixo caso contrário. **Defina as fronteiras** (ex.: 25,0% exato = Médio, > 25% = Alto), documente e teste exatamente nos limites 15/25/40.
5. `overallRisk` = pior sinal encontrado entre todas as posições e setores.
6. `recommendations`: uma string por regra violada, gerada a partir dos mesmos limiares (ex.: setor > 40% → "Reduzir exposição ao setor X (NN%)"). Sem violação → lista vazia ou mensagem de portfólio saudável (documento).

3.3 `GET /api/portfolios/{id}/rebalancing`

1. Peso atual por posição: `(valor_i / valorTotal × 100)`.
2. Normalizar `TargetAllocation` se `(Σ ≠ 100)`.
3. `deviation_i = currentWeight_i - targetWeight_i`; candidatos: `(|deviation| > 2%)`.
4. Delta financeiro: `(Δ_i = (targetWeight_i - currentWeight_i)/100 × valorTotal)`.
Negativo → SELL, positivo → BUY.
5. `quantity = floor(|Δ_i| / CurrentPrice)`; recalcular `(estimatedValue = quantity × CurrentPrice)`.
6. Descartar trades com `(estimatedValue < R$ 100)` **após** o arredondamento de quantidade.
7. `transactionCost = estimatedValue × 0,3%`; `totalTransactionCost = Σ`.
8. Ordenar por `(|deviation|)` desc (prioriza maiores desvios); `needsRebalancing = existe candidato`.
9. `expectedImprovement`: simular pesos pós-trade → recalcular concentração top-1 (ou top-3) → reportar a redução relativa: `"Redução de X% no risco de concentração"`.
Documentar a métrica escolhida.
10. Nota de consistência: como os pesos somam ~100 dos dois lados, `(Σ vendas ≈ Σ compras)` (rebalanceamento autofinanciado); o resíduo vem do arredondamento de quantidades — aceite e mencione no README. Otimização avançada (diferencial): casar vendas/compras de forma gulosa minimizando nº de trades cujo custo supere o benefício de redução de desvio.

4. Plano de execução em fases

Total estimado: **3h30–4h15**. Faça um commit ao fim de cada fase com mensagem clara — avaliadores olham o histórico do Git.

Fase 0 — Setup do projeto (15–20 min)

A estrutura obrigatória tem `Program.cs`, `Controllers/` e `SeedData.json` na **raiz do repositório**, com `Tests/` como subpasta. Para reproduzir literalmente:

bash

```
mkdir portfolio-analytics && cd portfolio-analytics
git init
dotnet new webapi --use-controllers -n PortfolioAnalytics -o .
dotnet new xunit -n PortfolioAnalytics.Tests -o Tests
dotnet add Tests/PortfolioAnalytics.Tests.csproj reference ./PortfolioAnalytics
dotnet new sln -n PortfolioAnalytics
dotnet sln add PortfolioAnalytics.csproj Tests/PortfolioAnalytics.Tests.csproj
dotnet add PortfolioAnalytics.csproj package Microsoft.EntityFrameworkCore.InMemory
dotnet add Tests/PortfolioAnalytics.Tests.csproj package FluentAssertions # o
```

⚠ **Projeto de teste aninhado:** como `Tests/` fica dentro da pasta do projeto API, o glob padrão do SDK tentaria compilar os testes junto. Adicione ao `PortfolioAnalytics.csproj`:

xml

```
<ItemGroup>
  <Compile Remove="Tests/**" />
  <Content Remove="Tests/**" />
  <None Remove="Tests/**" />
</ItemGroup>

<ItemGroup>
  <None Update="SeedData.json" CopyToOutputDirectory="PreserveNewest" />
</ItemGroup>
```

Criar pastas `Controllers/` `Services/` `Models/` `Models/DTOs/` `Data/`, colar o `SeedData.json` fornecido na raiz, conferir Swagger habilitado (template .NET 8 já traz Swashbuckle), `dotnet run` verde. **Commit: "chore: project scaffold"**.

Fase 1 — Modelos + Seed (30–35 min)

1. Abra o `SeedData.json` antes de escrever qualquer entidade e mapeie os nomes/tipos reais dos campos (não confie só na "estrutura simplificada" do enunciado — pode haver campos extras ou nomes diferentes).
2. Entidades `Portfolio`, `Position`, `Asset`, `PricePoint`, `MarketData` em `Models/`.
3. `Data/DataContext.cs` com os DbSets; `Data/DataSeeder.cs` que desserializa (`PropertyNameCaseInsensitive = true`) e popula.
4. `IMarketDataProvider` singleton com o snapshot de Selic/Ibovespa.

5. `Program.cs`: `builder.Services.AddDbContext(o => o.UseInMemoryDatabase("portfolio"))` + escopo de seed no startup.
6. Smoke test: log com contagens (`15 assets, 3 portfolios carregados`) ou endpoint temporário — confirme que nada veio zerado.

Commit: "feat: domain models, in-memory context and seed".

Fase 2 — Performance (50-60 min)

1. `Services/FinancialMath.cs`: `DailyReturns(prices)`, `SampleStdDev(values)`, `AnnualizeVolatility(daily)`, `AnnualizeReturn(totalReturnFraction, days)` — com comentários explicando cada fórmula (diferencial explícito do teste).
2. `IPerformanceCalculator` + `PerformanceCalculator` puro (recebe portfolio + assets + parâmetros).
3. DTOs (`PerformanceResponse`, `PositionPerformance`) espelhando o JSON da espec.
4. `AnalyticsController` com o primeiro endpoint + 404.
5. Validar no Swagger com os **3 portfólios** e conferir ordem de grandeza: retorno de 30 dias anualizado será um número grande (esperado — anote no README), volatilidade na casa de dois dígitos.

Commit: "feat: performance endpoint with financial math".

Fase 3 — Risk Analysis (40-50 min)

1. `IRiskAnalyzer` + implementação: Sharpe (reusando `FinancialMath/PerformanceCalculator`), concentração, setores, classificação via `AnalyticsRules`, recommendations geradas por regra.
2. DTOs + endpoint + 404.
3. Validar nos 3 portfólios: o "Crescimento" deve acusar concentração setorial; o "Conservador" deve sair Baixo/Médio.

Commit: "feat: risk analysis endpoint".

Fase 4 — Rebalancing (50-60 min)

1. `IRebalancingOptimizer` + implementação seguindo o algoritmo da seção 3.3 (normalização de targets, filtro 2%, floor de quantidade, mínimo R\$100, custo 0,3%, ordenação, expectedImprovement simulado).
2. DTOs + endpoint + 404.
3. Validar: o portfólio "Dividendos" (descrito como "precisa rebalanceamento") deve retornar `needsRebalancing: true` com trades coerentes; some manualmente um trade para conferir custo de 0,3%.

Commit: "feat: rebalancing suggestions endpoint".

Fase 5 — Testes (40-50 min)

Testar os **services puros** com dados construídos à mão (métodos helper/builders no projeto de teste). Use asserts com tolerância (`Assert.Equal(esperado, atual, precision: 2)` ou `BeApproximately`).

#	Teste	Cobre
1 	<code>TotalReturn_ComValoresValidos_CalculaCorretamente</code>	Obrigatório 1
2	<code>TotalReturn_InvestimentoZero_RetornaNull</code>	Edge divisão por zero
3 	<code>Volatility_ComHistoricoConhecido_RetornaDesvioAnualizado</code>	Obrigatório 2 (série pequena calculada à mão)
4	<code>Volatility_SemHistorico_RetornaNull</code>	Edge do FAQ
5 	<code>SharpeRatio_CenarioPositivoENegativo_CalculaCorretamente</code>	Obrigatório 3
6	<code>SharpeRatio_VolatilidadeZero_RetornaNull</code>	Edge divisão por zero
7 	<code>ConcentrationRisk_IdentificaMaiorPosicaoETop3</code>	Obrigatório 4
8	<code>RiskLevel_NasFronteiras_15_25_40_ClassificaConformeDocumentado</code>	Fronteiras das regras
9 	<code>Rebalancing_DesvioAcimaDe2_GeraTradeComQuantidadeECustoCorretos</code>	Obrigatório 5
10	<code>Rebalancing_DesvioMenorQue2_NaoSugereTrade</code>	Regra de limiar
11	<code>Rebalancing_TradeAbaixoDe100Reais_EhFiltrado</code>	Regra de mínimo
12	<code>TargetAllocation_NaoSoma100_EhNormalizada</code>	Decisão do FAQ

`dotnet test` verde. Commit: "test: unit tests for financial services".

Fase 6 — README + acabamento (25-30 min)

- 1. README (template na seção 6).
- 2. Passada de ILogger nos services logando valores intermediários dos cálculos (diferencial "logs estruturados para debug").
- 3. Comentários XML nas fórmulas do FinancialMath (diferencial explícito).
- 4. Prova final em ambiente limpo: git clone em outra pasta → dotnet run → 3 endpoints × 3 portfólios no Swagger → dotnet test.
- 5. Commit: "docs: README with technical decisions and formulas" → push na main.

Se o tempo apertar, corte diferenciais nesta ordem: testes de integração → otimização avançada de rebalanceamento → logs estruturados. Nunca corte os 5 testes obrigatórios, os edge cases nem o README.

5. Template do README

markdown

Portfolio Analytics API

Como executar

dotnet run # sobe em https://localhost:xxxx – Swagger em /swagger

Como testar

dotnet test

Endpoints

GET /api/portfolios/{id}/performance | /risk-analysis | /rebalancing

IDs disponíveis no seed: (listar os 3)

Decisões técnicas

- decimal p/ dinheiro, double p/ estatística (por quê)
- Services puros sem dependência de EF (testabilidade)
- EF Core InMemory + seed automático no startup
- Limiar de negócio centralizado em AnalyticsRules
- O que decidi NÃO fazer e por quê (arquitetura enxuta p/ o escopo de 3-4h)

Fórmulas financeiras

(tabela da seção 2.6: retorno, anualização base 365, vol amostral $\times \sqrt{252}$, Sharpe

Premissas adotadas

- "dias" = janela de 30 dias do histórico
- TargetAllocation $\neq$ 100% $\rightarrow$ normalizada proporcionalmente (com warning)
- Quantidades inteiras de ações; desvio residual aceito
- Vol do portfólio usa apenas ativos com histórico (cobertura parcial informada)
- Fallback de preço inválido: histórico $\rightarrow$ preço médio

Edge cases tratados

(lista: portfólio inexistente 404, divisões por zero, histórico vazio, symbol ó

6. Armadilhas que derrubam candidatos (checklist mental)

1. Usar o retorno em % dentro de $(1 + \text{TotalReturn})^{\dots}$ em vez de fração $\rightarrow$ número absurdo.
2. `Math.Pow` com `decimal` **não compila** $\rightarrow$ converter para `double` na estatística.
3. Sharpe com unidades mistas (retorno de 30 dias – Selic anual) $\rightarrow$ resultado sem sentido.
4. Volatilidade diária sem anualizar $\rightarrow$ 0,9 em vez de ~ 15 $\rightarrow$ avaliador estranha na hora.
5. Divisor `n` vs `n-1` no desvio-padrão sem documentar a escolha.
6. Divisões por zero não guardadas: `AveragePrice`, `TotalInvestment`, volatilidade no Sharpe.

7. Desserialização case-sensitive do JSON → tudo vem 0/null **silenciosamente**.
 8. Sugerir quantidade fracionária de ações ou trade abaixo de R\$100.
 9. Esquecer o 404 (é pergunta explícita do FAQ).
 10. Modificar o `SeedData.json` para "consertar" inconsistências.
 11. Services acoplados ao DbContext → testes viram sofrimento de mock (services puros resolvem).
 12. Repo que não roda em clone limpo (path absoluto do JSON, dependência não declarada).
-

7. Checklist final antes do push

- ☐ `dotnet run` funciona em clone limpo; Swagger abre
- ☐ 3 endpoints × 3 portfólios retornam JSON no formato da espec
- ☐ Portfólio inexistente → 404
- ☐ `dotnet test` verde com ≥ 5 testes (ideal 10-12)
- ☐ `SeedData.json` intacto na raiz
- ☐ Estrutura de pastas bate com a obrigatória (README, Controllers/, Services/, Models/, Tests/, Program.cs na raiz)
- ☐ README cobre: execução, testes, decisões, fórmulas, premissas
- ☐ Fórmulas comentadas no código
- ☐ Histórico de commits limpo, por fase
- ☐ Branch `main` atualizada e acesso concedido ao avaliador (se repo privado)